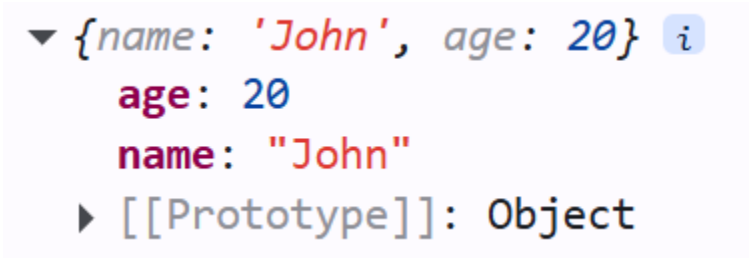
JS Prototype

1. **What is a Prototype in JavaScript?**
    a. In JavaScript, every object has a hidden property called **[[Prototype]]**, which is basically a reference to another object.
    b. OR
    c. In JavaScript, every function and object has its own property called Prototype.
    d. A prototype itself is also another object.
    e. So, the prototype has its prototype. This creates a *prototype chain*.
    f. Every JavaScript function (*except arrow functions*) has a special property called prototype.
    g. **For Example →**

```
let person = {
  name: "John",
  age: 20,
};

console.log(person);
```

Output:



2. Prototype Inheritance:

a. We can use the *prototype* to add properties and methods to a constructor function.

b. And objects inherit the properties and methods from a prototype.

c. **For Example →**

```
// Constructor Function
function Person(fName, lName) {
  this.fName = fName;
  this.lName = lName;
}
```

d. Now I want to add the '*age*' property to the Person Constructor function. But we can't directly add age to the Person.

```
// Adding a Property to the prototype
Person.age = 20; // ❌
```

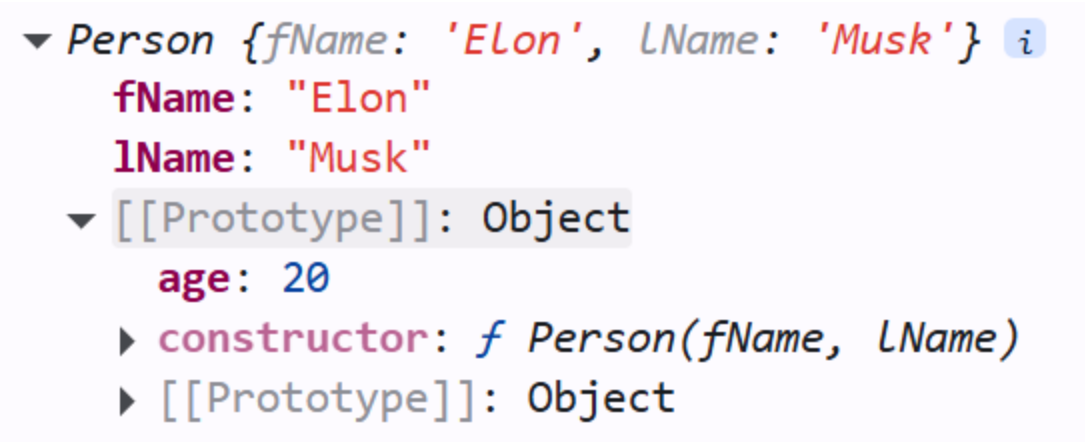
e. With the help of a prototype we can add the new property.

```
Person.prototype.age = 20;
```

f. Creating an object with the help of *new Person()*:

```
let person1 = new Person("Elon", "Musk");
console.log(person1);
```

g. **Output**: Here you can observe that the created age property is not present inside the Person1 object instead of it is present inside the prototype of object.



h. But you can access directly because it will inherit the property from the parent constructor.

i. First it will search the property inside itself, if it is not found then it will search inside the parent and inherit from the parent object.

```
console.log(person1.age); // 20
```

3. **Adding a method to prototype →**

```
// Adding a method to the prototype
Person.prototype.getFullName = function () {
  return this.fName + " " + this.lName;
};

console.log(person1.getFullName()); // Elon Musk
```

4. Changing Prototype Value:

a. For Example-1 →

```
// Constructor Function
function Person() {
  this.name = "Elon Musk";
}

Person.prototype.age = 20;

// Creating an object for Person constructor function
let person1 = new Person();
console.log(person1.age);

// Now lets change the value of the prototype property
Person.prototype.age = 25;

// Now I am creating person2 object
let person2 = new Person();
console.log(person2.age); //25
console.log(person1.age); //25
```

b. *What do you understand from the above example?*

i. *If you modify an existing prototype property, all existing and new objects see the updated value.*

c. For Example-2 →

```
function Person() {
  this.name = "Elon Musk";
}

Person.prototype.age = 20;

let person1 = new Person();
```

```
Person.prototype = { age: 25 }; //replaced prototype
object

let person2 = new Person();

console.log(person1.age); // 20 (old prototype)
console.log(person2.age); // 25 (new prototype)
```

d. *What do you understand from the above example?*

i. *If you replace the whole prototype object, only new objects will use the new one; old objects keep pointing to the old prototype.*

e. Final Clarification:

- *If you modify an existing prototype property*, all existing and new objects see the updated value.

- *If you replace the whole prototype object*, only new objects will use the new one; old objects keep pointing to the old prototype.

5. Prototype Chaining:

a. Prototype chaining is a mechanism in JavaScript where objects inherit properties and methods from other objects through their prototype.
b. Every object in JavaScript has an internal link (called **[[Prototype]]**) to another object, which forms a chain, and this is known as the prototype chain.
c. When you try to access a property or method on an object:
    i. JavaScript first looks for the property on the object itself.
    ii. If it doesn't find it, it checks the object's prototype.
    iii. If it still doesn't find it, it keeps checking the prototype of the prototype, and so on, until it reaches the end of the chain (*null*).